

Machine Learning Progress Handover (Updated)

Current Status

The **sklearn** module is 100% complete.

The **XGBoost** module is 100% complete.

The **LightGBM** module is 100% complete.

The next topic to begin is **CatBoost**.

Learning Style

- Explain **why** before **how**.
 - Teach intuition before syntax.
 - One concept at a time.
 - Complete runnable VS Code examples for every topic.
 - Professional software engineering practices throughout.
 - Reuse `prepData.py` whenever possible.
 - Build production-style projects rather than isolated notebook examples.
-

Current Project Structure

The project uses a shared `prepData.py` module which exports:

- `x`
- `y`
- `preprocessor`
- common sklearn imports
- XGBoost imports
- LightGBM imports

Each algorithm has its own standalone Python file.

Examples:

- `logisticRegression.py`
- `randomForest.py`
- `svm.py`
- `dbscan.py`
- `xgboostClassifier.py`
- `xgboostEarlyStopping.py`
- `xgboostTuning.py`

- lightgbmClassifier.py

No notebooks.

Completed Topics

Data Preprocessing

- SimpleImputer
 - StandardScaler
 - OneHotEncoder
 - ColumnTransformer
 - Pipelines
 - train_test_split
-

Supervised Learning

- Linear Regression
 - Logistic Regression
 - KNN
 - Decision Trees
 - Random Forests
 - Support Vector Machines
 - Gradient Boosting
-

Unsupervised Learning

- KMeans
 - KMeans++
 - DBSCAN
 - Agglomerative Clustering
-

Model Evaluation

Covered:

- Cross Validation
- GridSearchCV
- RandomizedSearchCV
- Nested Cross Validation

Metrics:

- Accuracy

- Precision
 - Recall
 - F1
 - Confusion Matrix
 - MAE
 - MSE
 - RMSE
 - R^2
 - Silhouette Score
-

Feature Engineering

Covered:

- SelectKBest
- Mutual Information
- Feature Importance
- Permutation Importance
- Polynomial Features

Understands:

- interaction terms
 - overfitting
 - degree selection
 - integration into pipelines
-

Model Persistence

Knows:

- `joblib.dump()`
- `joblib.load()`

Professional separation into:

- `train.py`
 - `predict.py`
-

Class Imbalance

Covered:

- `class_weight="balanced"`
- `stratify=y`
- `scoring="f1"`

Advanced sklearn

Custom Transformers

Implemented custom transformers using:

- BaseEstimator
- TransformerMixin

Implemented:

- RemoveCommas transformer

Understands:

- fit()
- transform()
- stateless transformers
- why fit() returns self
- X.copy()

Custom Estimators

Implemented custom estimators using:

- BaseEstimator

Understands:

- fit()
- predict()
- storing learned information in self
- sklearn estimator API

Pipeline Internals

Understands:

- named_steps
- get_feature_names_out()
- Pipeline.set_params()
- parameter naming

Examples:

- model__max_depth
- preprocessing__num__imputer__strategy

Synthetic Dataset Generation

Knows:

- `make_regression()`
- `make_classification()`
- `make_blobs()`

Knows when each should be used.

Miscellaneous Utilities

Covered:

- `Pipeline`
- `make_pipeline`
- `Pipeline.set_params()`
- `named_steps`
- `get_params()`

Knows why explicit Pipeline is preferred for production.

XGBoost

Completed:

- Why XGBoost exists
- Differences from sklearn Gradient Boosting
- `XGBClassifier`
- `XGBRegressor`
- `objective` parameter
- `eval_metric`
- Feature Importance
- Early Stopping
- Hyperparameter tuning
- Model persistence

Understands:

- regularization
- shrinkage
- learning rate
- smarter tree pruning
- native missing value handling
- feature importance
- early stopping workflow

- validation sets
- tuning using GridSearchCV / RandomizedSearchCV

Important engineering understanding:

The student now understands why XGBoost early stopping does **not** work cleanly with sklearn Pipelines.

They understand:

- Pipeline preprocesses only training data.
- eval_set is **not** automatically transformed.
- Therefore manual preprocessing is required for early stopping.
- Preprocessor is fit once on training data.
- Validation/Test data only use transform().
- XGBoost is then trained separately on processed matrices.

They understand the engineering reason rather than memorising the implementation.

LightGBM

Completed:

- LGBMClassifier
- LGBMRegressor
- num_leaves
- max_depth
- histogram-based learning
- GOSS
- EFB
- feature importance
- early stopping
- tuning

Understands:

- leaf-wise tree growth
- why num_leaves is the primary complexity control
- histogram binning for faster split finding
- GOSS (Gradient-based One-Side Sampling)
- EFB (Exclusive Feature Bundling)
- why LightGBM is typically faster than XGBoost on very large tabular datasets

Also understands:

- tiny datasets often require reducing:
 - min_child_samples
 - min_data_in_bin

to allow LightGBM to split.

Overall Understanding

The student already knew most ML theory from university.

This course has therefore focused primarily on:

- sklearn implementation
- engineering practices
- production workflows
- software architecture
- practical ML libraries

rather than teaching ML theory from scratch.

The student now naturally thinks in terms of:

Transformers

- fit()
- transform()

Estimators

- fit()
- predict()

and understands the sklearn API very well.

The student is now learning new libraries primarily through their engineering differences rather than their APIs.

Next Roadmap

1. CatBoost ★★★★★
 2. PyTorch
 3. CNNs
 4. RNNs / LSTMs
 5. Attention
 6. Transformers
 7. Hugging Face
 8. LLM Engineering
 9. MLOps
 10. TensorFlow/Keras (optional afterwards)
-

Immediate Next Lesson

Begin with **CatBoost**.

Cover:

- Why CatBoost exists
- Why categorical features are difficult
- Problems with OneHotEncoder
- Native categorical handling
- Ordered Target Encoding
- Ordered Boosting
- Target leakage
- CatBoostClassifier
- CatBoostRegressor
- Feature importance
- Early stopping
- Hyperparameter tuning
- Production implementation

Continue teaching in the same style as the sklearn, XGBoost and LightGBM modules.

The student already learns quickly at this stage. Focus on explaining the new concepts unique to CatBoost rather than repeating familiar estimator APIs or sklearn workflows.